



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИУ Информатика и системы управления»

КАФЕДРА «ИУ7 Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 5 по дисциплине «Анализ алгоритмов»

Тема Конвейерная обработка данных

Студент Еланская Е.Д.

Группа ИУ7-53Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Задание	4
2 Конструкторская часть	5
2.1 Описание алгоритма	5
3 Технологическая часть	6
3.1 Реализация	6
3.2 Тестирование	8
3.3 Пример лога программы	8
4 Исследовательская часть	9
4.1 Замеры реального времени	9
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11

ВВЕДЕНИЕ

Цель работы – сравнить последовательную и конвейерную обработку.

Задачи, необходимые для достижения поставленной цели:

- 1) описать и реализовать задание по варианту;
- 2) сравнить реальное время, затрачиваемое на обработку N заявок последовательно и конвейером.

1 Аналитическая часть

Конвейерная обработка — это метод повышения производительности, при котором сложная задача разбивается на последовательность простых этапов, и разные этапы выполняются одновременно над разными частями данных или инструкций. Конвейер обработки данных состоит из трёх основных компонентов: источника, одного или нескольких этапов обработки и пункта назначения. [2]

1.1 Задание

Конвейер из k потоков, включая 3 отдельных на каждое ОУ и $(k-3)$ дополнительно для ОУ2. k взять как рекомендованное значение для выбранной АЭВМ из ЛР4.

Обслуживающее устройство 1 выполняет считывание графа в формате матрицы смежности из текстового или бинарного файла (имя файла дано в заявке).

Обслуживающее устройство 2 выполняет обработку графа, согласно заданию на ЛР4, задействовав $k-3$ вспомогательных потоков.

Обслуживающее устройство 3 выполняет дамп ответа в файл, имя которого получено добавлением префикса к имени входного файла из заявки (файлы с ответами соотносятся с заявками 1:1).

3 нативных потока, которые передают друг другу информацию через очереди. Данные в 1-ю очередь (входную для 1-го обслуживающего устройства) в количестве N заявок заполнить перед пуском конвейера, считав их из файла (список имен файлов).

Вывод

В данном разделе мной был проведен анализ теоретических материалов, объясняющих принцип работы алгоритмов.

2 Конструкторская часть

2.1 Описание алгоритма

Схема алгоритма конвейерной обработки графов по заданию представлена на рис. 2.1.



Рисунок 2.1 — Схема алгоритма конвейерной обработки.

Вывод

В данном разделе мной были описан алгоритм работы конвейера по заданию.

3 Технологическая часть

Для реализации алгоритмов использовался язык программирования C++17, так как он поддерживает статическую типизацию, имеет возможность отключения сборщика мусора и поддерживает нативную многопоточность. Отладка происходила в среде разработки «Visual Studio 2022».

3.1 Реализация

```
1 void task_1(std::queue<request>& queue_1, std::queue<request>& queue_2, std::mutex& m2,
2   std::condition_variable& cv12, bool& d12) {
3   request rtmp;
4   while (!queue_1.empty()) {
5       rtmp = queue_1.front();
6       queue_1.pop();
7       rtmp.add_timestamp();
8       read_to_matr(rtmp);
9       {
10          std::lock_guard<std::mutex> lk(m2);
11          queue_2.push(rtmp);
12          cv12.notify_one();
13      }
14      {
15          std::lock_guard<std::mutex> lk(m2);
16          d12 = true;
17      }
18      cv12.notify_all();
19  }
20
21 void task_2(std::queue<request>& queue_2, std::queue<request>& queue_3, std::mutex& m2,
22   std::mutex& m3,
23   std::condition_variable& cv12, std::condition_variable& cv23, bool& d12, bool& d23) {
24   request rtmp;
25   while (true) {
26       {
27          std::unique_lock<std::mutex> lk(m2);
28          cv12.wait(lk, [&]{ return !queue_3.empty() || d12; });
29          if (queue_2.empty() && d12)
30              break;
31          else {
32              rtmp = queue_2.front();
33              queue_2.pop();
34          }
35          rtmp.add_timestamp();
36          find_components(rtmp);
37          {
38              std::lock_guard<std::mutex> lk(m3);
39              queue_3.push(rtmp);
40              cv23.notify_one();
41          }
42      }
43      {
44          std::lock_guard<std::mutex> lk(m3);
```

```

45     d23 = true;
46 }
47 cv23.notify_all();
48 }
49
50 void task_3(std::queue<request>& queue_3, std::mutex& m3, std::condition_variable& cv23,
    bool& d23,
51     std::map<std::chrono::system_clock::time_point, std::vector<int>>& log) {
52     request rtmp;
53     while (true) {
54         {
55             std::unique_lock<std::mutex> lk(m3);
56             cv23.wait(lk, [&] { return !queue_3.empty() || d23; });
57             if (queue_3.empty() && d23)
58                 break;
59             else {
60                 rtmp = queue_3.front();
61                 queue_3.pop();
62             }
63         }
64         rtmp.add_timestamp();
65         dump_to_file(rtmp);
66         create_log(rtmp, log);
67     }
68 }
69
70 int conveyer(const std::string& filename) {
71     std::queue<request> queue_1, queue_2, queue_3;
72     int res = fill_requests(filename, queue_1);
73     if (res == 0) {
74         std::condition_variable cv12, cv23;
75         bool d12 = false, d23 = false;
76         std::mutex m2, m3;
77         std::map<std::chrono::system_clock::time_point, std::vector<int>> log; // timestamp :
            [id, i]
78         std::vector<std::thread> threads(3);
79         threads[0] = std::thread(task_1, std::ref(queue_1), std::ref(queue_2), std::ref(m2),
            std::ref(cv12), std::ref(d12));
80         threads[1] = std::thread(task_2, std::ref(queue_2), std::ref(queue_3), std::ref(m2),
            std::ref(m3), std::ref(cv12), std::ref(cv23), std::ref(d12), std::ref(d23));
81         threads[2] = std::thread(task_3, std::ref(queue_3), std::ref(m3), std::ref(cv23),
            std::ref(d23), std::ref(log));
82         for (int i = 0; i < 3; i++) {
83             if (threads[i].joinable())
84                 threads[i].join();
85         }
86         log_to_file(log);
87     }
88     return res;
89 }

```

Листинг 1 – Код программы.

Таблица 3.1 — Результаты тестирования.

Вход	Выход
requests_test.txt	graph/res_graph_11.txt graph/res_graph_6.txt graph/res_graph_5.txt graph/res_graph_12.txt graph/res_graph_7.txt graph/res_graph_13.txt graph/res_graph_9.txt graph/res_graph_14.txt graph/res_graph_8.txt graph/res_graph_10.txt log.txt

3.2 Тестирование

3.3 Пример лога программы

1	0	0	2025-12-26,12:30:55
2	1	0	2025-12-26,12:30:55
3	2	0	2025-12-26,12:30:55
4	3	0	2025-12-26,12:30:55
5	4	0	2025-12-26,12:30:55
6	5	0	2025-12-26,12:30:55
7	6	0	2025-12-26,12:30:55
8	7	0	2025-12-26,12:30:56
9	0	1	2025-12-26,12:30:56
10	1	1	2025-12-26,12:30:56
11	0	2	2025-12-26,12:30:56
12	2	1	2025-12-26,12:30:56
13	1	2	2025-12-26,12:30:56
14	3	1	2025-12-26,12:30:56
15	2	2	2025-12-26,12:30:56
16	3	2	2025-12-26,12:30:56
17	4	1	2025-12-26,12:30:56
18	5	1	2025-12-26,12:30:56
19	4	2	2025-12-26,12:30:56
20	5	2	2025-12-26,12:30:56
21	6	1	2025-12-26,12:30:56
22	7	1	2025-12-26,12:30:56
23	6	2	2025-12-26,12:30:56
24	7	2	2025-12-26,12:30:56

Листинг 2 – Лог программы для 8 заявок.

Вывод

В данном разделе мной была реализована программа по заданию и проведено её успешное тестирование.

4 Исследовательская часть

4.1 Замеры реального времени

Замеры проводились на ноутбуке Redmibook Pro 14 2022:

- процессор Intel i7-12650H (архитектура x86-64, 10 ядер по 2.3 ГГц);
- оперативная память 16 Гб (DDR4, 5200 МГц);
- операционная система Windows 11 Home 25H2.

Для замеров использовалась функция `now()` из библиотеки `chrono` [1, с. 705].

Значение $k=16$ (получено в лабораторной работе №4).

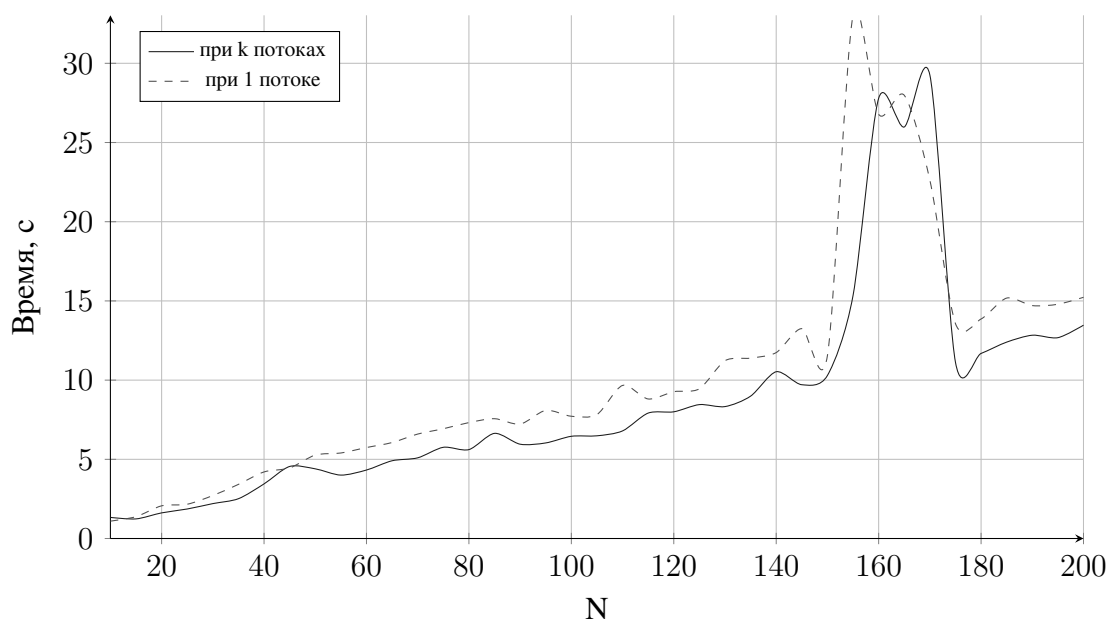


Рисунок 4.1 — График зависимости времени от N заявок.

Вывод

В данном разделе мной был проведен сравнительный анализ времени выполнения при различном количестве нативных потоков.

ЗАКЛЮЧЕНИЕ

В ходе работы мной были описаны и реализован алгоритм конвейерной обработки. Также мной было проведено сравнение временных характеристик последовательной и конвейерной обработок данных. Конвейерная обработка затрачивает меньше реального времени, чем последовательное выполнение той же задачи.

Таким образом, все задачи были выполнены, а поставленная цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. C++17 standard (ISO/IEC DIS 14882:2017). [Электронный ресурс] Режим доступа: <https://web.archive.org/web/20170325025026/http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4660.pdf> (дата обращения: 20.11.2025).
2. Конвейерная организация. Режим доступа: https://citforum.ru/hardware/svk/glava_5.shtml (дата обращения: 24.12.25)